CEG 4350/5350 – Operating Systems Internals and Design

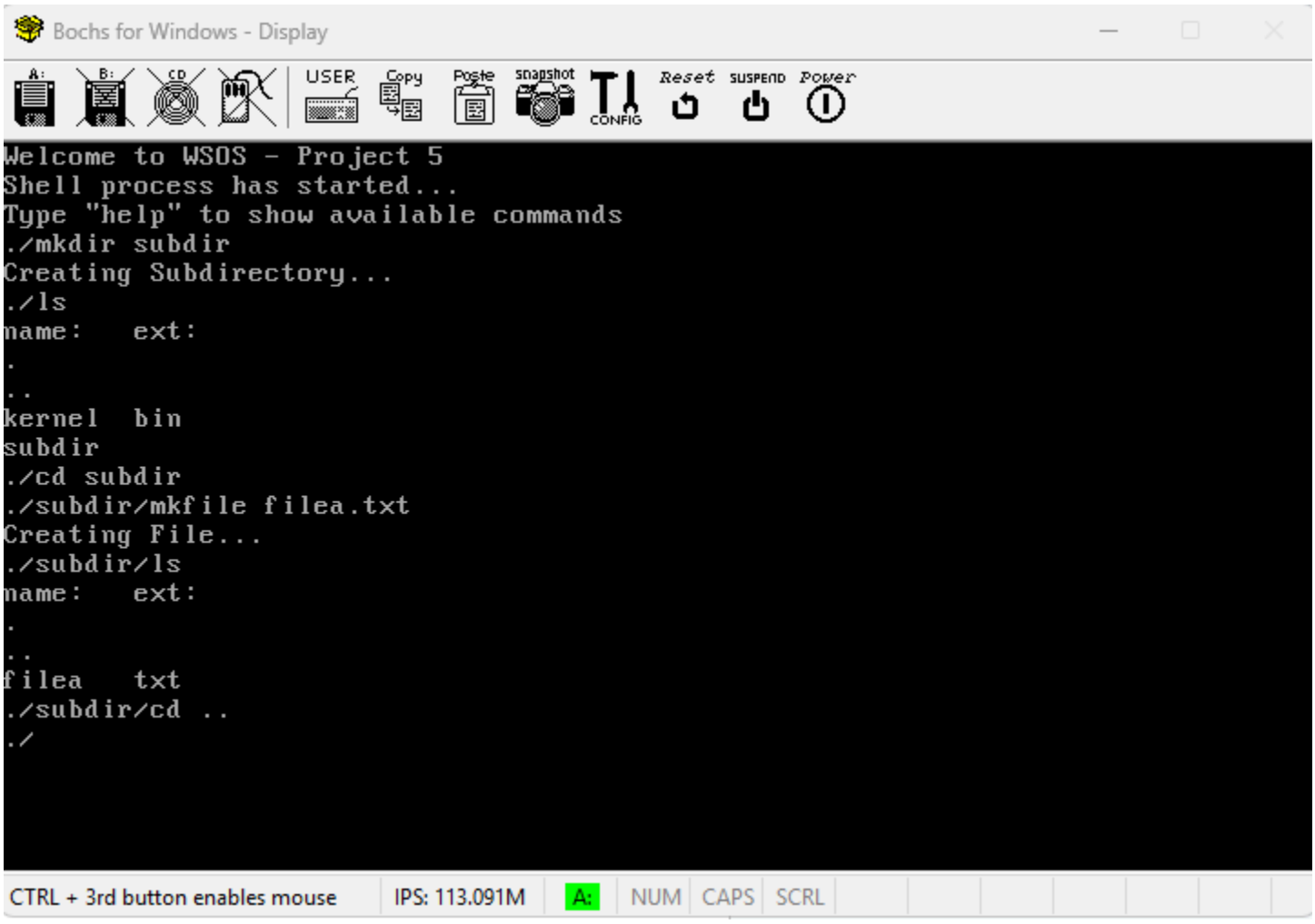
Project 5 – Subdirectories

Learning Objectives

- Learn how to implement subdirectories in the FAT file system.

Overview

In this project you will implement creating subdirectories, navigating to subdirectories, and adding files to subdirectories. This builds upon our previous work to implement the FAT file system. Creating a new subdirectory is as simple as creating a new directory entry inside of the parent directory. Next, create an entry for ".." that points to the parent directory (use reserved cluster number 0 for root) and an entry "." that points to the subdirectory itself. Creating a subdirectory in the OS and analyzing the disk, the results should look like this:



Creating a subdirectory in the root, navigating to it, creating a file inside of it, and navigating back to root


FAT Table
Root Directory
Root directory cluster = 0 (reserved)
"subdir" Subdirectory Data
"filea.txt" File Data

FAT Directory Operations

To implement basic directory operations in our OS make the following modifications:

1. Modify project5.c:
    a. int createSubdirectory(char *subdirectoryName)
        i. Checks if the currentDirectory is opened
            1. If currentDirectory is not opened, print an error message, return non-zero value
        ii. Find the next free directory entry inside of the currentDirectory
            1. Loop through currentDirectory.startingAddress until a directory_entry_t is found with filename[0] == 0
        iii. Check to see if the directory that you are creating this subdirectory in needs to be resized
            1. Note: the root directory cannot be resized, if its full return non-zero
            2. Hint: you can reuse some sections of createFile() to do this
        iv. Find the next free cluster in the FAT and mark it as EOF
            1. Hint: use nextFreeCluster() and fat0->clusters[next] = 0xFFFF;
            2. Next, write the FATs to disk using writeFATs()
        v. Using the free entry found in step ii:
            1. Set filename to the subdirectoryName argument
            2. Set extension to “ ”
            3. Set attributes to 0x10 (labeled as a subdirectory)
            4. Set starting cluster to the next free cluster found in step iv
            5. Set the file size to 512
            6. Write the currentDirectory to the disk using writeCurrentDirectory()
        vi. Create a buffer for the new sector for the subdirectory and add:
            1. Add an entry for “.” containing the directory entry for the subdirectory itself
                a. uint8 buffer[512] = {0};
                b. directory_entry_t *selfEntry = (directory_entry_t *)buffer;
                c. Dereference selfEntry and overwrite with entry from step v, then use stringcopy to overwrite filename with “. “
            2. Add an entry for “..” containing the directory entry for the currentDirectory
                a. directory_entry_t *parentEntry = ((directory_entry_t *)buffer) + 1;
                b. Dereference parentEntry and overwrite with currentDirectory.directoryEntry, then use stringcopy to overwrite filename with “.. “
            3. Write the buffer to the disk:
                a. floppy_write(0, next + CLUSTER_TO_SECTOR, (void *) buffer, 512);
        vii. Return 0 if no errors have occurred in any of the previous steps

b. int openDirectory(char *directoryName)
    i. Checks if the currentDirectory is opened
        1. If currentDirectory is not opened, print an error message, return non-zero value
    ii. Checks if the currentFile is opened
        1. If currentFile is opened, print an error message, return non-zero value
    iii. Find the directory entry that has the name directoryName
        1. Loop through currentDirectory.startingAddress until a directory_entry_t is found with filename equal to directoryName and attributes with the 0x10 bit set
        2. Hint: you can reuse some sections of openFile() to do this
    iv. Checks if we are opening the root directory (i.e.: directoryEntry->startingCluster == 0)
        1. If we are opening the root directory, set currentDirectory.directoryEntry to root directory entry:
            a. currentDirectory.directoryEntry = &rootDirectoryEntry;
        2. Read the root directory from the disk:
            a. readFailed = floppy_read(0, ROOT_DIR_SECTOR, (void *)currentDirectory.startingAddress, ROOT_SIZE);
    v. If we are opening a non-root directory:
        1. Load the directory off of the disk by traversing the FATs from the directory entry's startingCluster found in step iii
            a. Read each 512 byte cluster into currentDirectory.startingAddress + 512 * index
        2. Once the directory has been loaded into memory, search for the selfEntry that has filename "." and set currentDirectory.directoryEntry = selfEntry;
    vi. Return 0 if no errors have occurred in any of the previous steps

How to Submit

To submit your code, commit your changes to your GitHub repository and include a link to your repository in the Pilot dropbox.

Grading

This lab is worth 5.00 points, distributed as follows:

Task	Points
createSubdirectory() creates a directory entry of the subdirectory in the parent	1.00
createSubdirectory() creates a directory entry of the parent in the subdirectory	1.00
openDirectory() allows for opening a subdirectory	1.00
openDirectory() allows for opening a parent directory	1.00
Files can be created inside of subdirectories	1.00
Total	5.00